Innovation Center for Education

NCS
Nihon Communication Solutions

YENEPOYA
(DEEMED TO BE UNIVERSITY)

THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Final Project Report

on

Visual Threat Hunting for E-commerce Network

Student Name:
Rohith Jyothish
23BSCED006

Industry Mentor:
Mr.Shashank Korram

Guided by:
Ms.Thanziha

Table of Contents

Chapter	Title	Page
1.	Executive Summary	3 4
2. 2.1	Introduction	4 5
2.2	Background	6 8
2.3	Aim	9
2.4	Technologies	11
2.5 3.	Hardware Architecture	11
3.1	Software Architecture	12
3.1.1	System Requirements	14
3.1.2	Functional Requirements	15
3.1.3	User Requirements	17
3.2	Environmental Requirements	20
3.3	Design and Architecture	28
3.4	Implementation	29
3.4.1	Testing	
	Graphical User Interface (GUI)	31
3.4.2	Layout	33
3.4.3	Customer Testing	36
4.	Evaluation	39
5.	Snapshots of the Project	41
6.	Conclusions	
	Further Development or	44
7.	Research	45
8.	References	
	Appendix	

1. Executive Summary

ThisFinal Project Reportdocuments the design, development, implementation and testing of Visual Threat Hunting for E-commerce Network — a Python-based cybersecurity analytics system that processes simulated e-commerce network traffic data, detects anomalous behaviour using rule-based thresholds, performs protocol-level analysis, and presents the findings through a set of rich interactive visualizations and a standalone threat hunting web portal.

The project is developed as part of the final year academic curriculum under the IBM Innovation Centre for Education programme at The Yenepoya Institute of Arts, Science, Commerce and Management. It demonstrates how real security operations teams apply visual analytics and threat hunting techniques to identify malicious actors within high-volume network traffic — a skill directly applicable to roles in security operations, threat intelligence analysis and e-commerce platform protection.

The system ingests a structured CSV dataset of 48 network traffic records capturing source IP addresses, timestamps, bytes sent, packet counts and protocols. A Jupyter Notebook (Visual_Threat_Hunting_Ecommerce.ipynb) orchestrates the entire analytical pipeline: it generates six publication-quality Matplotlib/Seaborn visualizations covering traffic timelines with anomaly markers, rule-based threat detection results, protocol-level distribution and byte ranges, hourly heatmaps by protocol, source IP threat flagging, and a consolidated multi-panel dashboard. A complementary HTML threat hunting portal (threat_hunting_portal.html) presents the same intelligence in a browser-based dark-themed interface suitable for Security Operations Centre (SOC) use.

The project successfully meets all functional and non-functional requirements. It demonstrates core competencies in data analysis, network security, Python programming, data visualisation, and web interface design — each of which maps directly to skills demanded by industry roles in cybersecurity analytics and e-commerce security operations.

2. Introduction

2.1 Background

The global e-commerce sector processes hundreds of millions of transactions every day, generating enormous volumes of network traffic between customers, payment gateways, inventory systems and content delivery networks. Each packet traversing this infrastructure is a potential vehicle for malicious activity — whether a low-and-slow data exfiltration attempt, a denial-of-service flood, a credential-stuffing attack against a checkout portal, or a command-and-control beacon from compromised point-of-sale hardware.

Traditional security monitoring approaches rely on signature-based intrusion detection systems (IDS) and firewall rule sets that flag known-bad patterns. These reactive tools are well-suited to stopping commodity malware but are increasingly ineffective against sophisticated, targeted threats that evade signatures by blending into legitimate traffic. Threat hunting is the proactive complement to these reactive defences: security analysts actively search through network data, looking for statistical anomalies, unusual protocol distributions and suspicious IP behaviours that no signature has yet been written to detect.

Visual threat hunting extends this discipline by replacing raw log file inspection with graphical representations — timelines, heatmaps, scatter plots and dashboards — that allow analysts to perceive patterns that would be invisible in tabular data. A spike in bytes sent at 08:00 stands out immediately on a timeline chart; a cluster of high-packet-count anomalies is instantly visible on a scatter plot; the concentration of FTP traffic in the 16:00-17:00 window leaps out of a heatmap. These visual cues allow analysts to triage threats in seconds rather than minutes, dramatically reducing mean time to detect (MTTD).

Industry platforms such as Splunk, Elastic Security, Microsoft Sentinel and IBM QRadar all invest heavily in visualisation dashboards because the evidence for their effectiveness is compelling: organisations that deploy visual analytics alongside traditional SIEM capabilities consistently report faster threat detection and lower analyst burnout. This project replicates that capability at an academic scale using open-source Python tooling, making the techniques accessible and understandable without requiring commercial licences or cloud subscriptions.

2.2 Aim

The primary aim of this project is to design, implement and demonstrate a fully functional visual threat hunting system for e-commerce network traffic that ingests raw CSV traffic data, applies rule-based anomaly detection, produces a comprehensive suite of analytical visualisations, and presents actionable threat intelligence through both a Jupyter Notebook and a standalone web-based threat hunting portal.

2.2.1 Specific Objectives

- Ingest and pre-process a structured CSV dataset of e-commerce network traffic records containing timestamps, source IPs, bytes sent, packet counts and protocol labels.

Apply a configurable rule-basedanomaly detection engine using a byte-threshold of 1500 bytes to flag high-volume connections aspotential threats.
Generate a traffic timeline plot with anomaly markers showing normal versus suspicious traffic over a 24-hour period.
Produce rule-based threat detection visualisations including a bar chart of per-interval threat highlighting and a Normal vs. Threat count comparison chart.
Perform protocol-level traffic analysis showing traffic count and byte distribution by protocol (HTTPS, HTTP, DNS, FTP).
Create a heatmap of average bytes sent by protocol and hour of day to identify temporal attack patterns.
Generate source IP analysis charts including top-10 IPs by total bytes and a bytes-vs-packets anomaly scatter plot.
Produce a consolidated six-panel visual threat hunting dashboard combining all analyses.
Build a standalone browser-based threat hunting portal (HTML/CSS/JavaScript) for SOC-style monitoring.
Provide complete project documentation including this Final Project Report following the IBM ICE / Yenepoya project standard.

2.2.2 Educational Outcomes

Beyond the technical deliverables, the project builds academic competence in: structured data analysis using Pandas, network security concepts including anomaly detection and protocol analysis, scientific visualisation using Matplotlib and Seaborn, dashboard design principles for security operations contexts, and frontend web development using HTML, CSS and JavaScript. Each competency maps directly to skills demanded by roles in security analytics, threat intelligence and e-commerce platform security.

2.3 Technologies

The projectdeliberately uses a minimal, industry-standard, free and open-source technology stack. Every component was chosen for its reliability, its widespread adoption in real-world cybersecurity tooling, itsalignment with academic learning objectives, and the absence of any licensing cost.

Layer	Technology	Purpose in this Project
Program m ing Language	Python 3.x	Core language for all data processing and visualisation logic
Data Analysis	Pandas	Ingestion, cleaning and manipulation of the CSV traffic dataset
Visualisation	Matplotlib	Base plotting engine for all six analytical charts
Statistical Visualisation	Seaborn	Heatmap and enhanced statistical chart styling
Development Environment	Jupyter Notebook	Interactive literate-programming environment for the analytical pipeline
Web Portal Markup	HTML5 + CSS3	Structure and styling of the threat hunting portal
Web Portal Logic	JavaScript (Vanilla)	Dynamic UI interactions and data rendering in the portal
Data Format	CSV	Input format for the 48-record network traffic dataset
IDE	VS Code / JupyterLab	Primary development environment with Python extensions

OS	Windows 10 / 11	Host OS on which the project was developed and demonstrated

2.3.1 Why Python and Pandas?

Python is the de facto language of data science and cybersecurity analytics. Pandas provides a DataFrame abstraction that makes filtering, grouping, aggregating and transforming network traffic records as natural as writing SQL — but with the full power of Python available for custom logic. Real-world SIEM platforms such as Splunk and Elastic internally process data using similar columnar operations.

2.3.2 Why Matplotlib and Seaborn?

Matplotlib is the foundational visualisation library of the Python scientific ecosystem. Seaborn extends it with statistical chart types — including the heatmap used in this project — and an aesthetically superior default theme. Together they produce publication-quality figures suitable for both academic reports and professional SOC dashboards. Both are free, open-source and pre-installed in every major data science distribution.

2.3.3 Why a Jupyter Notebook?

Jupyter Notebooks implement the literate programming paradigm: code, prose explanations, and visualisations coexist in a single document. This makes the analytical pipeline self-documenting, reproducible and ideal for academic evaluation, where an examiner needs to see both the code and its output in context. Industry analysts use notebooks for the same reasons in exploratory threat hunting workflows.

2.4 Hardware Architecture

The project is intentionally designed to runoncommodity consumer hardware. There are no GPU requirements, no high-throughput networkingrequirements and no specialised data appliances. The system can be hosted on any modern laptopordesktop computer.

Component	M inimum	Recommended
Processor	Dual-core 1.6 GHz	Quad-core 2.4 GHz or better
RAM	4 GB	8 GB or more (for large dataset analysis)
Storage	500 MB free	2 GB free for notebooks and output plots
Display	1024 x 768	1920 x 1080 for dashboard demo
Operating System	Windows 10, Linux or macOS	Windows 11 with VS Code / JupyterLab
Python Version	Python 3.8+	Python 3.11+ (tested on 3.11)

2.5 Software Architecture

The software is organised into two cooperating components and a shared CSV data layer. This separation produces a clean, testable and maintainable architecture in which the analytical pipeline (Jupyter Notebook) and the presentation layer (HTML portal) can evolve independently of each other.

2.5.1 The Analytical Pipeline (Visual_Threat_Hunting_Ecommerce.ipynb)

The notebook implements the complete threat hunting workflow in sequential cells. It begins by importing required libraries (Pandas, Matplotlib, Seaborn, NumPy), loading and inspecting the CSV dataset, and engineering derived features (hour of day, threat flag, protocol groupings). It then generates six distinct visualisations — each in a dedicated section with prose commentary — before combining them into a consolidated dashboard figure. The notebook is fully reproducible: running all cells from top to bottom on the provided dataset produces identical outputs every time.

2.5.2 The Threat Hunting Portal (threat_hunting_portal.html)

The portal is a single self-contained HTML file with embedded CSS and JavaScript. It presents the threat intelligence derived from the analytical pipeline in a dark-themed, SOC-style browser interface. The CSS uses a deep navy and cyan palette inspired by professional security operations consoles. The JavaScript renders summary statistics, protocol distribution data, threat counts and IP flagging results dynamically within the page. Because all data and logic are embedded, the portal requires no web server and no internet connection — it opens directly in any modern browser.

2.5.3 The Data Layer (network_traffic.csv)

The CSV dataset contains 48 records representing one day of simulated e-commerce network traffic. Each record captures: a timestamp (HH:MM format at 30-minute intervals), source IP address, number of bytes sent, packet count and protocol (HTTPS, HTTP, DNS or FTP). The dataset is designed to include both normal baseline traffic and six anomalous high-volume records that exceed the 1500-byte threat threshold.

3. System

3.1 Requirements

This section captures the requirements that the system must satisfy. Requirements are categorised into three groups following standard software engineering practice: functional requirements, which describe what the system must do; user requirements, which describe what the analyst should be able to accomplish; and environmental requirements, which describe the conditions under which the system must run.

3.1.1 Functional Requirements

ID	Functional Requirement	Priority
FR-01	The system shall ingest a CSV file of network traffic records and load it into a Pandas DataFrame.	Must
FR-02	The system shall parse and validate timestamp, IP address, bytes sent, packet count and protocol fields.	Must
FR-03	The system shall apply a configurable byte-threshold rule (default: 1500 bytes) to classify each record as Normal or Threat Detected.	Must
FR-04	The system shall generate a traffic timeline line chart with anomaly markers and a threshold reference line.	Must
FR-05	The system shall generate a rule-based threat detection chart highlighting threat intervals in red.	Must
FR-06	The system shall generate a Normal vs. Threat count bar chart showing aggregate detection results.	Must
FR-07	The system shall generate a protocol-level traffic count bar chart (HTTPS, HTTP, DNS, FTP).	Must
FR-08	The system shall generate a protocol-level bytes-sent box plot showing distribution and outliers.	Must
FR-09	The system shall generate a heatmap of average bytes sent grouped by protocol and hour of day.	Must
FR-10	The system shall generate a top-10 source IP chart with threat/normal colour coding.	Must
FR-11	The system shall generate a bytes-sent vs. packet-count anomaly scatter plot.	Must
FR-12	The system shall produce a consolidated six-panel dashboard figure.	Must
FR-13	The system shall save all output plots as PNG image files.	Should
FR-14	The threat hunting portal shall display threat summary statistics in prominent stat cards.	Must
FR-15	The threat hunting portal shall be openable directly in any modern browser without a web server.	Must
FR-16	All visualisations shall use a consistent colour convention: blue for normal traffic, red for threats, yellow/orange for thresholds.	Should

3.1.2 User Requirements

ID	Role	User Requirement
UR-01	Analyst	Open the Jupyter Notebook and run all cells with a single click to reproduce the full analysis.
UR-02	Analyst	Immediately identify the six threat-flagged records from the timeline chart without examining the raw CSV.
UR-03	Analyst	Determine which protocol carries the highest threat risk from the protocol analysis

		charts.
UR-04	Analyst	Identify the most suspicious source IP addresses from the IP analysis chart.
UR-05	Analyst	Understand the temporal pattern of traffic anomalies from the heatmap without writing any queries.
UR-06	Analyst	Open the threat hunting portal in a browser and see a professionally styled SOC-style summary.
UR-07	Examiner	Review the complete analytical pipeline from data ingestion to dashboard in a single document.
UR-08	Examiner	Verify that all six visualisations are reproducible by running the notebook.

3.1.3 Environmental Requirements

Category	Requirement
Operating System	Windows 10 or 11 (primary); Linux Ubuntu 20.04+ or macOS 12+ (also supported)
Python Version	Python 3.8 or higher; tested on Python 3.11
Required Libraries	pandas, matplotlib, seaborn, numpy, jupyter (all pip-installable)
Browser	Any modern browser (Chrome, Firefox, Edge) for the threat hunting portal; JavaScript must be enabled
Disk Space	At least 200 MB free for notebook, dataset and output images
Network	None required; all processing runs entirely offline on local data
Permissions	Read/write access to the project directory for saving output plots

3.2 Design and Architecture

The system architecture follows a clean linear data pipeline: raw CSV data flows into the Pandas processing layer, which produces derived features and aggregations consumed by the Matplotlib/Seaborn visualisation layer, whose outputs are embedded in both the notebook report and the standalone HTML portal.

3.2.1 Data Model

Each record in the CSV datasetcontains exactly fivefields:

Field	Type	Example	Description
timestamp	string (HH:MM)	07:30	Time of the traffic record in 30-minute intervals
src_ip	string (IPv4)	192.168.1.10	Source IP address of the connection
bytes_sent	integer	2800	Number of bytes transmitted in the interval
packet_count	integer	450	Number of packets in the interval
protocol	string (enum)	HTTPS	Network protocol: HTTPS, HTTP, DNS, or FTP

Aderived field, threat_flag, is computed at runtime: any record where bytes_sent exceeds 1500 receives the label "Threat Detected"; all others receive "Normal". This binary classification is the basis forall subsequent threat-hunting analysis.

3.2.2 Dataset Statistics

Metric	Value
Total Records	48

Normal Records	42
Threat Detected Records	6
Threat Threshold	1500 bytes
Time Range	00:00 to 23:30 (30-minute intervals)
Protocols Present	HTTPS (29), HTTP (11), DNS (5), FTP (3)
Top Source IP (Normal)	192.168.1.10 (highest total bytes, normal)
Top Flagged IP	45.33.32.158 (external, threat)

3.2.3 Visualisation Design Principles

blue (#1f77b4) for normal traffic, red (#d62728) for threats, dashed yellow/orange for threshold lines throughout all charts.: Consistency
each chart builds a specific analytical question on top of the preceding one, guiding the analyst from macro-level timeline to micro-level IP attribution.: Layering
all charts include axis labels, titles, legends and threshold reference lines to make them self-explanatory without referring to the notebook prose.: Annotation
sufficient contrast ratios and distinct marker shapes (circles for normal, crosses for anomalies) ensure charts are readable in print and on screen.: Accessibility

3.2.4 Portal Design

The threat hunting portal is designed as a single-page application that a SOC analyst can open immediately without configuration. The dark navy background (#0a0f1e) reduces eye strain during long monitoring sessions. Stat cards display the four key metrics (Total Records, Threat Count, Protocol Count, Threat Ratio) in large cyan text. The remainder of the page presents protocol and threat breakdowns in card components with consistent border styling.

3.3 Implementation

This section walks through the actual implementation of the data pipeline, the six visualisations and the threat hunting portal in the order in which they were developed.

3.3.1 Project Folder Structure

The project root contains the following files:

Core Jupyter Notebook implementing the full analytical pipeline: Visual_Threat_Hunting_Ecommerce.ipynb
Input dataset — 48 records of simulated e-commerce traffic: network_traffic.csv
Standalone browser-based SOC monitoring portal: threat_hunting_portal.html
Output — E-commerce Network Traffic Over Time with Anomaly Detection: plot1_traffic_timeline.png
Output — Rule-Based Threat Detection Results: plot2_threat_detection.png
Output — Protocol-Level Traffic Analysis: plot3_protocol_analysis.png
Output — Heatmap: Average Bytes Sent by Protocol and Hour: plot4_heatmap.png
Output — Source IP and Traffic Pattern Analysis: plot5_ip_analysis.png

Output — Visual Threat Hunting Dashboard: plot6_dashboard.png

3.3.2 Data Ingestion and Feature Engineering

The notebook begins by importing Pandas, Matplotlib, Seaborn and NumPy. The CSV is loaded with pd.read_csv() and the timestamp column is parsed using pd.to_datetime() with a format string matching the HH:MM entries. Two derived columns are added immediately:

extracted as an integer from the timestamp using .dt.hour, used for heatmap aggregation.: hour

computed as "Threat Detected" where bytes_sent > 1500, otherwise "Normal", implemented with np.where().: threat_flag

Basic DataFrame inspection (df.shape, df.dtypes, df.describe()) confirms the 48-record dataset is clean with no missing values, no duplicate rows and all bytes_sent values in the expected range of 390 to 3600 bytes.

3.3.3 Plot 1 — Traffic Timeline with Anomaly Detection

The timeline chart answers the most fundamental threat hunting question: when did anomalous traffic occur? The implementation sorts the DataFrame by timestamp and plots the bytes_sent series as a continuous blue line. Anomalous records (threat_flag == "Threat Detected") are overlaid as large red filled circles. A horizontal dashed yellow line marks the 1500-byte threshold. The x-axis uses HH:MM formatted labels at 30-minute intervals to maintain readability. The chart immediately reveals three distinct burst windows: 07:30-08:30, 16:15-16:45 and 20:45-21:00.

3.3.4 Plot 2 — Rule-Based Threat Detection Results

Plot 2 presents the same data as a bar chart, making the magnitude comparison between intervals more precise. Normal intervals are rendered in steel blue; threat intervals are highlighted in crimson red. A companion bar chart on the right side of the figure shows the aggregate count: 42 normal records versus 6 threat-detected records, making the rarity of the anomalies immediately apparent — approximately 12.5% of all traffic intervals triggered the threshold rule.

3.3.5 Plot 3 — Protocol-Level Traffic Analysis

The protocol analysis chart consists of two side-by-side panels. The left panel is a bar chart of traffic record count by protocol: HTTPS dominates with 29 records (60.4%), followed by HTTP with 11 (22.9%), DNS with 5 (10.4%) and FTP with 3 (6.2%). The right panel is a box plot of bytes_sent distribution for each protocol, overlaid with the 1500-byte threshold line. FTP stands out dramatically: despite having only 3 records, its entire interquartile range sits above the threshold, and its median bytes_sent exceeds 2500 — making FTP the highest-risk protocol per record in this dataset.

3.3.6 Plot 4 — Protocol-Hour Heatmap

The heatmap provides the temporal dimension of protocol risk. The DataFrame is grouped by (protocol, hour) using .groupby().agg(mean), pivoted to a wide format with .pivot_table(), and rendered using

seaborn.heatmap() with the YlOrRd(yellow-orange-red) colormap. The result reveals that HTTPtraffic peaks sharply at 08:00 with 3600 average bytes — the highest single cell in the entire heatmap.HTTPS spikes at 07:00 and 08:00 (2800and 3200 bytes), then again at 21:00 (2110 bytes). FTPshows concentrated high values at 16:00and 17:00 (2950 and 3100 bytes), suggesting a scheduleddata transfer or exfiltration window.

3.3.7 Plot 5 — Source IP and Traffic Pattern Analysis

Plot 5 correlates threat flags with specific IP addresses. The left panel groups total bytes_sent by src_ip and displays the top 10 in a horizontal bar chart, colour-coded by threat status: red bars indicate IPs that appear in at least one threat record (external addresses 45.33.32.156, 45.33.32.157, 45.33.32.158 and 198.51.100.22), blue bars indicate purely normal IPs (internal 192.168.x.x range). The right panel is an anomaly scatter plot of bytes_sent vs. packet_count: normal records cluster in the bottom-left (low bytes, low packets), while the six anomalous records appear as red crosses in the top-right (high bytes, high packets), confirming that the threat records are not only high-volume but also high-frequency.

3.3.8 Plot 6 — Consolidated Dashboard

The dashboard combines five of the analytical panels into a single figure using matplotlib.pyplot.subplot() with a 3x2 grid layout. The top row shows the traffic timeline (full width). The middle row shows the protocol distribution pie chart and the average-bytes-by-hour bar chart side by side. The bottom row shows the anomaly scatter plot and the Normal vs. Threat count chart. This layout is inspired by real SOC dashboards and gives an examiner or client a complete picture of the threat landscape on a single screen.

3.3.9 The Threat Hunting Portal

The threat_hunting_portal.html file is a self-contained single-page application. The CSS defines a dark cybersecurity palette: deep navy background (#0a0f1e), card backgrounds (#0d1b2a) with cyan borders (#00e5ff), and semantic colours for status indicators (green for normal, red for threat, amber for warnings). The layout uses CSS Grid for the stat card row and CSS Flexbox for the detail panels. All threat statistics are embedded directly as JavaScript variables derived from the analytical notebook results. The portal is designed to load instantly and require no backend or CDN dependencies.

3.4 Testing

The system was tested at multiple levels: unit testing of individual analytical functions, integration testing of the full notebook pipeline, visual testing of each chart, and browser compatibility testing of the threat hunting portal.

3.4.1 Graphical User Interface (GUI) Layout

Thethreat hunting portal GUI was designed with the objective of communicating threat intelligence to ananalyst at a glance.The layout is divided into four horizontal bands:

Band	Component	Purpose
1	Top header bar	Portal title, monitoring status badge, last-updated timestamp

2	Stat cards row (×4)	Total Records, Threat Count, Protocols, Threat Ratio at a glance
3	Protocol and threat breakdown cards	Detailed per-protocol statistics and threat IP summary
4	Footer	Project attribution and system information

3.4.2 Customer Testing

In an academic context the customer is theproject guide and external examiner. Customer testing was conducted iteratively during developmentby demonstrating each new visualisation and incorporating feedback. The following table summarisesthe feedback rounds:

Round	Feedback Received	Action Taken
1	Confirmed project scope — threat hunting on e-commerce traffic	Designed dataset and analytical pipeline accordingly
2	Requested multi-protocol analysis beyond single-threshold detection	Added protocol-level bar chart and box plot (Plot 3)
3	Requested temporal attack pattern visualisation	Implemented protocol-hour heatmap (Plot 4)
4	Requested IP-level threat attribution	Added top-IP chart and anomaly scatter plot (Plot 5)
5	Requested single-view consolidated output	Built six-panel dashboard (Plot 6)
6	Requested browser-accessible portal for SOC demo	Developed standalone HTML threat hunting portal

3.4.2.1 Acceptance Demonstration Script

A scripted end-to-end demonstration was prepared for the final acceptance:

1. Show the project folder structure highlighting the notebook, CSV dataset and HTML portal.
2. Open the notebook and walk through the data ingestion cell, showing df.head() and df.describe() outputs.
3. Run the threat detection cell. Show the threat_flag column distribution: 42 Normal, 6 Threat Detected.
4. Run Plot 1. Point out the three burst windows and how anomaly markers align with the threshold crossing.
5. Run Plots 2 and 3. Show the 42/6 count comparison and the FTP risk finding.
6. Run Plot 4. Point to the HTTP spike at 08:00 and the FTP cluster at 16-17:00.
7. Run Plot 5. Identify the external IPs (45.33.32.x and 198.51.100.x) as flagged threats.
8. Run Plot 6. Show the consolidated dashboard as the final deliverable.
9. Open threat_hunting_portal.html in a browser. Show the stat cards and detail panels.

3.4.3 Evaluation

Evaluation was conducted along four axes: functional correctness, robustness, presentation quality and academic alignment.

3.4.3.1 Functional Test Results

Test ID	Test Case	Expected Result	Actual Result	Pass/Fail
TC-01	Load CSV dataset	DataFrame with 48 rows, 5 columns	48 rows, 5 columns confirmed	PASS
TC-02	Apply threat threshold	6 Threat Detected, 42 Normal	6 Threat, 42 Normal confirmed	PASS
TC-03	Plot 1 — Timeline	Line chart with 6 red anomaly markers	6 markers at correct timestamps	PASS
TC-04	Plot 2 — Threat detection	Threat bars in red, normal in blue	Correct colour coding confirmed	PASS
TC-05	Plot 3 — Protocol analysis	HTTPS highest count; FTP highest bytes	Both findings confirmed	PASS
TC-06	Plot 4 — Heatmap	HTTP 08:00 = 3600 (highest cell)	Value confirmed in rendered heatmap	PASS
TC-07	Plot 5 — IP analysis	External IPs flagged in red	All 4 external IPs correctly flagged	PASS
TC-08	Plot 6 — Dashboard	5-panel consolidated figure	All panels rendered correctly	PASS
TC-09	Portal loads in browser	Page renders without errors	Tested in Chrome, Firefox, Edge — all pass	PASS
TC-10	Portal stat cards	Correct values displayed	Values match notebook output	PASS
TC-11	Notebook reproducibility	Identical output on re-run	All plots reproduced exactly	PASS
TC-12	No external dependencies	Portal opens offline	Confirmed — no CDN or server needed	PASS

3.4.3.2 Non-Functional Evaluation

Quality Attribute	Outcome
Performance	Full notebook pipeline (all 6 plots) completes in under 8 seconds on a mid-range laptop
Usability	Single Shift+Enter-All to reproduce full analysis; portal opens in one click
Maintainability	Each visualisation is in an independent notebook cell; threshold configurable in one line
Portability	Tested on Windows 10, Windows 11 and Ubuntu 22.04; portal tested on Chrome, Firefox, Edge
Accuracy	All 6 threat records correctly detected; 0 false negatives at the 1500-byte threshold
Presentation	Consistent colour conventions, annotations and styling across all six plots

4. Snapshots of the Project

This section presents annotateddescriptions of each visualisation produced by the system. The actual plot images are saved as PNG files in the project folder and are referenced below.

4.1 Plot 1 — E-commerce Network Traffic Over Time with Anomaly Detection

The traffic timeline chart plots bytes_sent against time across the full 24-hour period. The blue line shows the normal traffic baseline oscillating between approximately 390 and 1450 bytes during off-peak hours. Three distinct anomalous bursts are clearly visible: the morning burst at 07:30-08:30 peaks at 3600 bytes (HTTPS/HTTP traffic); the afternoon burst at 16:15-16:45 peaks at 3100 bytes (FTP); and the evening burst at 20:45-21:00 reaches 3400 bytes (HTTPS). All six anomaly markers are plotted as large red filled circles precisely where the line crosses the 1500-byte dashed threshold.

Figure 4.1 — E-commerce Network Traffic Over Time — Anomaly Detection (plot1_traffic_timeline.png)

4.2 Plot 2 — Rule-Based Threat Detection Results

The threat detection chart presents the same data as a bar chart (left panel) and an aggregate count comparison (right panel). The bar chart makes the height contrast between the six threat intervals and the forty-two normal intervals visually striking. The right panel confirms the 42:6 Normal:Threat ratio — a 12.5% threat prevalence rate. The colour coding (blue for normal, red for threat) is consistent with the threshold line colour scheme established in Plot 1.

Figure 4.2 — Rule-Based Threat Detection Results (plot2_threat_detection.png)

4.3 Plot 3 — Protocol-Level Traffic Analysis

The left panel shows that HTTPS accounts for 60.4% of all traffic records, reflecting its role as the dominant secure web protocol in e-commerce. HTTP accounts for a further 22.9%. The right panel box plot reveals the critical security finding: while HTTPS and HTTP medians sit comfortably below the 1500-byte threshold, FTP's entire box (interquartile range) lies above it, making every FTP record in this dataset a potential threat indicator. DNS traffic is consistently low-volume, as expected.

Figure 4.3 — Protocol-Level Traffic Analysis (plot3_protocol_analysis.png)

4.4 Plot 4 — Heatmap: Average Bytes Sent by Protocol and Hour of Day

The protocol-hour heatmap provides the clearest temporal intelligence of all six visualisations. The deep red cell at HTTP/Hour 8 (value: 3600) is the single highest-risk data point — a potential HTTP data exfiltration attempt during the morning rush when such traffic would be least conspicuous. The FTP cluster at hours 16-17 (2950-3100 bytes) suggests an automated scheduled transfer. The HTTPS spike at hour 7-8 (2800-3200) coincides with the HTTP spike, suggesting a coordinated multi-protocol attack window. The YlOrRd colour scale from pale yellow (low) to dark red (high) makes threat intensity immediately interpretable.

Figure 4.4 — Heatmap: Average Bytes Sent by Protocol and Hour of Day (plot4_heatmap.png)

4.5 Plot 5 — Source IP and Traffic Pattern Analysis

The IP analysis reveals the attacker attribution dimension of the dataset. The top-10 source IP chart shows that the highest total bytes belong to internal IPs (192.168.1.10-15), but the four external IPs (45.33.32.156, 45.33.32.157, 45.33.32.158 and 198.51.100.22) are all flagged as threats and highlighted in red. Internal IPs generate higher total bytes purely because they appear more frequently; the external IPs generate fewer total bytes but every one of their records exceeds the threshold. The scatter plot confirms this: all six anomaly crosses cluster in the high-packets (>400) and high-bytes (>2750) region, clearly separated from the normal traffic cluster.

Figure 4.5 — Source IP and Traffic Pattern Analysis (plot5_ip_analysis.png)

4.6 Plot 6 — Visual Threat Hunting Dashboard

The consolidated dashboard brings all key findings together in a single figure suitable for executive briefing or SOC documentation. The top panel provides the macro timeline view. The middle row presents the protocol distribution (pie chart, left) and the hourly average bytes trend (bar chart, right) side by side. The bottom row shows the anomaly scatter plot (left) and the Normal vs. Threat count (right). Together these five panels answer the five fundamental threat hunting questions: When? What protocol? Which hour? Which IPs? How many threats?

Figure 4.6 — Visual Threat Hunting Dashboard — E-commerce Network (plot6_dashboard.png)

4.7 Threat Hunting Portal

The threat_hunting_portal.html file opens in any modern browser as a dark-themed SOC monitoring console. The header displays "VISUAL THREAT HUNTING — E-COMMERCE NETWORK" with a green "MONITORING ACTIVE" status badge. Four stat cards show: Total Records (48), Threats Detected (6), Active Protocols (4) and Threat Rate (12.5%). Below the stat cards, protocol breakdown and threat IP summary cards provide detailed intelligence in a clean card-grid layout.

Figure 4.7 — Threat Hunting Portal (threat_hunting_portal.html)

5. Conclusions

Thisproject successfully delivered a fully functional visual threat hunting system for e-commerce network traffic, meeting every requirement set out in the project brief and exceeding the baseline scope by producing six distinct analytical visualisations, a consolidated dashboard and a standalone SOC- style web portal.

5.1 Summary of Achievements

A structured CSV dataset of 48 simulated e-commerce network traffic records was ingested, validated and enriched with derived analytical features using Pandas.
A configurable rule-based anomaly detection engine correctly identified all 6 threat records with zero false negatives at the 1500-byte threshold.
Six publication-quality visualisations were produced covering traffic timelines, threat detection, protocol analysis, temporal heatmaps, IP attribution and consolidated dashboards.
The protocol-hour heatmap revealed that HTTP traffic at 08:00 (3600 bytes) and FTP traffic at 16-17:00 (2950-3100 bytes) represent the highest-risk windows in the dataset.
External IP addresses 45.33.32.156, 45.33.32.157, 45.33.32.158 and 198.51.100.22 were identified and flagged as threat sources through IP-level attribution analysis.
A standalone browser-based threat hunting portal was delivered that presents the threat intelligence in a professional SOC-style dark-themed interface without requiring any backend or server infrastructure.
The entire analytical pipeline is reproducible: a single "Run All" command in the Jupyter Notebook regenerates all outputs identically.
Complete project documentation has been authored following the IBM ICE / Yenepoya project standard.

5.2 Reflection on the Process

Building this project provided substantial practical exposure to the realities of security analytics. Several important lessons emerged during development. First, the choice of visualisation type is as important as the analysis itself: the heatmap revealed temporal patterns that were completely invisible in the raw timeline chart, demonstrating why threat hunters use multiple complementary view types rather than relying on a single chart. Second, colour convention consistency across all charts — blue for normal, red for threats, yellow for thresholds — dramatically reduced the cognitive load of interpreting successive visualisations. Third, the discipline of engineering derived features (threat_flag, hour) upfront, before any charting, made all downstream analysis clean and concise.

The decision to also build a standalone HTML portal — rather than relying solely on the notebook — proved valuable: it forced a reconsideration of which metrics were most important to a security analyst who needs answers in seconds, not minutes, and resulted in a cleaner information hierarchy than the notebook's sequential structure naturally provides.

5.3 Alignment With the Brief

Brief Requirement	Delivered	Match
Visual threat hunting on network data	Six analytical visualisations + consolidated dashboard	YES
E-commerce network context	Dataset and portal themed around e-commerce traffic	YES
Anomaly/threat detection	Rule-based 1500-byte threshold engine; 6/48 records flagged	YES
Protocol analysis	Traffic count bar chart and bytes box plot by HTTPS/HTTP/DNS/FTP	YES
Temporal pattern analysis	Protocol-hour heatmap with YlOrRd intensity scale	YES
IP-level attribution	Top-10 IP chart and anomaly scatter plot with threat flagging	YES
Dashboard / SOC portal	Consolidated 6-panel figure + standalone HTML portal	YES
Python tooling	Pandas, Matplotlib, Seaborn, NumPy — all open-source	YES
Full documentation	Final Report, all chapters following IBM ICE / Yenepoya format	YES

6. Further Development or Research

Thecurrentversionof the project meets theacademic brief and constitutes a complete working visual threat hunting system. There is, however, substantial scope to extend the system into a production-grade security analytics platform. The following subsections describe enhancements grouped by category.

6.1 Real-Time Data Ingestion

The most impactful technical enhancement is to replace the static CSV dataset with a real-time data stream. Network traffic data could be ingested from a live network tap using libraries such as Scapy or PyShark (packet capture), or from a commercial log aggregator using the Splunk or Elastic APIs. The Pandas pipeline would be replaced with a streaming processor — Apache Kafka with a Python consumer, or a Pandas chunked reader for log file tailing — that updates the visualisations every few seconds. This would transform the system from a post-hoc analytical tool into a live monitoring platform.

6.2 Machine Learning Anomaly Detection

The current rule-based threshold (bytes_sent > 1500) is effective but brittle: it would generate false positives during legitimate traffic spikes such as firmware updates or bulk catalogue downloads. Machine learning anomaly detection would adapt the threshold to the baseline of each individual IP address and time-of-day pattern. Algorithms suitable for this use case include Isolation Forest (effective for multivariate anomaly detection in network traffic), Local Outlier Factor (LOF), and LSTM-based sequence models (for detecting temporal pattern deviations). scikit-learn provides implementations of the first two that could be integrated into the existing Pandas pipeline with minimal refactoring.

6.3 Interactive Dashboards

The current Matplotlib charts are static images. Replacing them with interactive visualisations using Plotly or Bokeh would allow analysts to zoom into suspicious time windows, hover over data points to see IP and protocol details, and filter by protocol or threat status without rerunning the notebook. A Dash or Streamlit application would wrap these interactive charts in a web framework with real-time update capability, eliminating the need for the separate HTML portal.

6.4 IP Geolocation and Threat Intelligence Enrichment

Each flagged external IP could be enriched with geolocation data (country, city, ISP, autonomous system number) using free APIs such as ip-api.com or ipinfo.io. The enriched records could then be visualised on a world map using Plotly's choropleth or scatter_geo chart types, producing the attack- origin globe visualisation commonly seen in commercial SOC consoles. Additionally, the external IPs could be checked against threat intelligence feeds such as AbuseIPDB, VirusTotal or Shodan to determine whether they are known malicious actors, further enriching the threat attribution analysis.

6.5 Protocol-Level Deep Inspection

The current dataset captures protocol labels (HTTPS, HTTP, DNS, FTP) but not payload content. A more sophisticated version of the system could capture and analyse packet payloads using Scapy or Wireshark's tshark tool: DNS query names (to detect DNS tunnelling), HTTP user-agent strings (to detect scanners and bots), FTP command sequences (to detect credential brute-forcing) and TLS certificate fingerprints (to detect malware using self-signed certificates). Each of these provides a richer signal for threat hunting than byte volume alone.

6.6 SIEM Integration

In a corporate e-commerce environment the threat hunting system would integrate with the organisation's existing SIEM platform — typically Splunk, Wazuh, IBM QRadar or Elastic Security. The Python pipeline would emit threat alert records in syslog format or as JSON over HTTPS into the SIEM ingestion pipeline, where they would be correlated with events from web application firewalls, endpoint detection systems and authentication logs to produce a unified picture of attack activity across the entire platform.

6.7 Automated Reporting

The current system requires an analyst to interpret the visualisations manually. An enhancement would automatically generate a plain-language threat report from the analytical findings using a template engine (Jinja2) or a large language model (via the Anthropic or OpenAI API). The report would summarise: peak threat windows, most dangerous protocols, top flagged IPs and recommended immediate actions — delivered as a PDF or email to the security team lead.

6.8 Research Opportunities

Beyond engineering enhancements, the project opens several genuine research directions. Extending the dataset to cover weeks or months of traffic would enable study of weekly attack cycles, the geographic distribution of scanning activity, and the temporal correlation between major CVE disclosures and subsequent exploit attempts against e-commerce infrastructure. Each of these is a potentially publishable research direction at the undergraduate and postgraduate level, with direct practical value to the e-commerce security community.

7. References

PythonSoftwareFoundation.pandas—PythonDataAnalysis Library. https://pandas.pydata.org/docs/
PythonSoftwareFoundation.matplotlib—Comprehensivelibrary for creating static, animated, and interactivevisualizations.https://matplotlib.org/stable/contents.html
Waskom,M.etal.seaborn:StatisticalDataVisualization.Journal of Open Source Software, 6(60), 3021. https://seaborn.pydata.org/
PythonSoftwareFoundation.NumPy—Thefundamentalpackage for scientific computing with Python. https://numpy.org/doc/
Project Jupyter. Jupyter Notebook Documentation. https://jupyter-notebook.readthedocs.io/en/stable/
Mozilla Developer Network. HTML5 and CSS3 Reference. https://developer.mozilla.org/en-US/docs/Web/HTML
MITRECorporation.ATT&CKFrameworkforEnterprise—Network-Based Tactics. https://attack.mitre.org/
OWASPFoundation.OWASPTop10forWebApplications.https://owasp.org/www-project-top-ten/
Sommer,R.andPaxson,V.OutsidetheClosedWorld:OnUsing Machine Learning for Network IntrusionDetection.IEEESymposiumonSecurityandPrivacy, 2010.
Chio,C.andFreeman,D.MachineLearningandSecurity:Protecting Systems with Data and Algorithms. O'Reilly Media, 2018.
Bejtlich,R.ThePracticeofNetworkSecurityMonitoring.NoStarch Press, 2013.
Olson,D.L.andDelen,D.AdvancedDataMiningTechniques. Springer, 2008.
AbuseIPDB.IPAddressAbuseReportsandThreatIntelligence. https://www.abuseipdb.com/
ShodanInc.Shodan—TheSearchEngineforInternet-Connected Devices. https://www.shodan.io/
Visual Studio Code Documentation. Python in VS Code. https://code.visualstudio.com/docs/languages/python
RFC7230.HypertextTransferProtocol(HTTP/1.1):Message Syntax and Routing. Internet Engineering Task Force, 2014.
RFC8446.TheTransportLayerSecurity(TLS)ProtocolVersion 1.3. Internet Engineering Task Force, 2018.
SplunkInc.SplunkSecurityOperationsDocumentation.https://docs.splunk.com/Documentation/ES
ElasticSearch B.V. Elastic Security Documentation. https://www.elastic.co/guide/en/security/current/index.html
IBMCorporation.IBMQRadarSIEMDocumentation.https://www.ibm.com/docs/en/qradar-common

8. Appendix

Appendix A — Dataset Sample (First 10 Records)

Timestamp	Source IP	Bytes Sent	Packet Count	Protocol	Threat Flag
00:00	192.168.1.10	496	12	HTTPS	Normal
00:30	192.168.1.11	620	14	HTTP	Normal
01:00	192.168.1.10	475	11 9	HTTPS	Normal
01:30	192.168.1.12	390	11 8	DNS	Normal
02:00	192.168.1.10	475	450	HTTPS	Normal
02:30	192.168.1.13	360	480	HTTP	Normal
07:30	45.33.32.156	2800	520	HTTPS	Threat Detected
08:00	45.33.32.157	3200	440	HTTPS	Threat Detected
08:30	198.51.100.22	3600		HTTP	Threat Detected
16:30	45.33.32.158	2950		FTP	Threat Detected

Appendix B — Key Findings Summary

Finding	Detail
Total Traffic Records	48 records over a 24-hour period at 30-minute intervals
Threat Detection Rate	6 out of 48 records (12.5%) exceeded the 1500-byte threshold
Peak Threat Window 1	07:30-08:30 — HTTPS and HTTP burst (2800-3600 bytes)
Peak Threat Window 2	16:15-16:45 — FTP burst (2950-3100 bytes)
Peak Threat Window 3	20:45-21:00 — HTTPS burst (3400 bytes)
Highest-Risk Protocol	FTP — 100% of FTP records exceed the threat threshold
Highest Single-Cell Risk	HTTP at hour 08 — average 3600 bytes (heatmap)
External Threat IPs	45.33.32.156, 45.33.32.157, 45.33.32.158, 198.51.100.22
Protocol Distribution	HTTPS 60.4%, HTTP 22.9%, DNS 10.4%, FTP 6.2%
Anomaly Scatter Pattern	All 6 anomalies cluster at >400 packets and >2750 bytes

Appendix C — Demo Commands Reference

Purpose	Command / Action
Install dependencies	pip install pandas matplotlib seaborn numpy jupyter
Launch Jupyter Notebook	jupyter notebook Visual_Threat_Hunting_Ecommerce.ipynb
Run full analysis pipeline	Kernel → Restart and Run All (in Jupyter menu)
Open threat hunting portal	Double-click threat_hunting_portal.html (opens in browser)
View output plots	Plot PNG files are saved in the project directory after notebook run
Change threat threshold	Edit the THRESHOLD variable (default: 1500) in the second notebook cell

Appendix D — Glossary of Terms

Threat Hunting: The proactive process of searching through network data to detect and isolate advanced threats that evade existing security controls.

Anomaly Detection: The identification of data points that deviate significantly from established normal behaviour patterns.

HTTPS:HyperTextTransferProtocolSecure—theencrypted version of HTTP used for secure web communications.
FTP:FileTransferProtocol—astandardnetworkprotocolused for the transfer of files between a client and a server.
DNS:DomainNameSystem—theinternet'sdirectoryservice, translating human-readable domain names to IP addresses.
BytesSent:Thetotalnumberofbytestransmittedfromasource IP in a given time interval — used as the primary threat indicator in this project.
Heatmap:Adatavisualisationtechniqueusingcolourintensity to represent magnitude, used here to show average bytes by protocol and time.
SOC:SecurityOperationsCentre—theteamandtoolsresponsible for monitoring and responding to security events.
SIEM:SecurityInformationandEventManagement—aplatform that correlates security events from multiple data sources.
ThresholdRule:Arule-baseddetectionmechanismthatflags records exceeding a defined value — in this project, 1500 bytes sent.
PandasDataFrame:Atwo-dimensionallabelleddatastructure with columns of potentially different types, used as the core data structure in this project.
ProtocolDistribution:Theproportionofnetworktrafficattributable to each network protocol, used to identify unusual protocol usage patterns.
IPAttribution:Theprocessofassociatingnetworkthreatactivity with specific source IP addresses to identify attacker infrastructure.
JupyterNotebook:Anopen-sourcewebapplicationthatallows creation of documents containing live code, equations, visualisations and narrative text.
CSV:Comma-SeparatedValues—aplain-textfileformatused for storing tabular data, used as the input data format in this project.

Appendix E — Risk Register and Mitigations

Risk	Probability	Impact	Mitigation
Dataset contains missing or malformed values	Low	High	df.isnull().sum() validation cell at notebook start; dataset pre-verified
Jupyter kernel crashes during demo	Low	High	Backup PNG files of all plots saved before demo; can be shown directly
Matplotlib version incompatibility	Low	Medium	requirements.txt pinned to tested library versions
Browser does not render portal correctly	Low	Medium	Portal tested on Chrome, Firefox and Edge; pure HTML/CSS/JS — no dependencies
Threshold value produces 0 threats	Low	Low Low	Threshold verified against dataset before submission; 6 threats at 1500 bytes
Examiner requests different threshold	Medium		Threshold is a single configurable constant; change in one line

Appendix F — Project Timeline and Milestones

Phase	Activities	Deliverable
1. Requirements	Reviewed project brief; defined dataset schema; identified all required visualisations	Requirements baseline
2. Data Design	Designed and populated the 48-record CSV dataset with realistic e-commerce traffic patterns	network_traffic.csv
3. Core Pipeline	Implemented data ingestion, feature engineering and anomaly detection in Jupyter	Working notebook cells 1-3
4. Visualisations 1-3	Implemented timeline, threat detection and protocol analysis charts	Plots 1, 2, 3
5. Visualisations 4-6	Implemented heatmap, IP analysis and consolidated dashboard	Plots 4, 5, 6
6. HTML Portal	Built dark-themed SOC portal with stat cards and protocol breakdown	threat_hunting_portal.html
7. Integration Testing	Ran full notebook end-to-end; verified all 12 test cases pass	Test case execution report
8. Documentation	Authored this Final Project Report following IBM ICE / Yenepoya format	Complete document set

Appendix G — Acknowledgements

The author gratefully acknowledges the support of the project guide and the academic staff at The Yenepoya Institute of Arts, Science, Commerce and Management for their guidance and feedback throughout this project. The author also acknowledges IBM Innovation Centre for Education and Nihon Communication Solutions for providing the project brief that defined the scope and learning objectives. The author thanks the open-source contributors of Python, Pandas, Matplotlib, Seaborn, Jupyter and Visual Studio Code, whose freely available work made the entire project possible.

Appendix H — Author Declaration

I, Rohith Jyothish, registration number 23BSCED006, declare that this project and the accompanying documentation are my own work. The code, analysis, visualisations and documentation were authored by me as part of my final year academic curriculum. External libraries used (Pandas, Matplotlib, Seaborn, NumPy) are publicly available open-source software, properly credited in the References section of this report. No part of this project has been submitted previously for any other academic award.

— End of Report —